Kako nam rojevi insekata iz prirode pomazu da resimo probleme u racunarstvu, Zoe Perić

Sadržaj

1. Ant Colony Optimization 2
   Model potrage ACO 2
      osnovne faze kroz koje prolaze veštacki mravi: 2
      ACO rešava probleme u računarstvu. 2
2. Artificial Bee Colony 3
   Model selekcije ABC 3
      Vrste ABC: 3
      ABC rešava probleme u računarstvu: 3
3. Dragonfly Algorithm 4
         Model kretanja DA 4
      Vrste DA: 4
      DA rešava probleme u računarstvu: 4
4. Firefly Algorithm 4
   Model prati intenzitet svetlosti FA 4
      FA rešava probleme u računarstvu : 4
5. Grasshopper Optimization 5
   Model inteligencije roja 5
      GOA rešava probleme u računarstvu : 5
6. Bug 5
7. Algoritmi 6
   1)ACO: 6
   2)ABC 6
      a) Peaks 7
      b) Rastrignin 7
      c) Easom 7
      d) Beale 8
   3)DA 8
   4)FA 8
   5)GOA 9

1. Ant Colony Optimization

Model potrage ACO

npr: ponašanje mrava u potrazi za hranom, podela rada u mravljoj kloniji, sortiranje larvi, transport tereta.

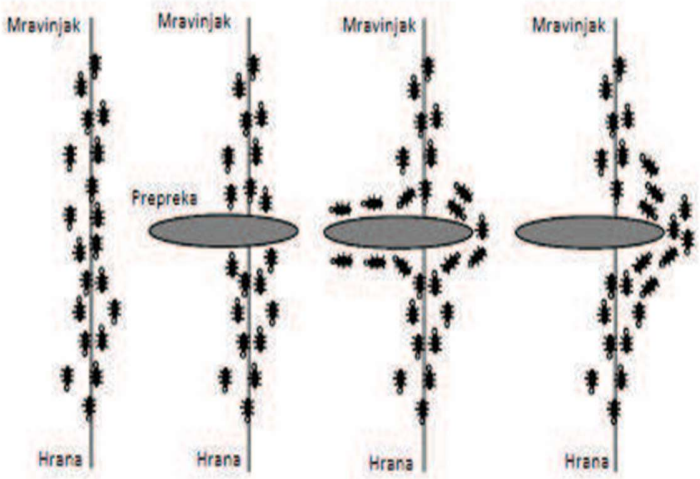
Prvi ACO algoritam je predstavljen 1990-ih godina. To je metaheuristička tehnika ze rešavanje složenih problema.

Dok se mravi kreću po putu oni ostavljaju **feromone** kako bi označili put koji će do hrane pratiti ostali članovi kolonije.

Cilj je da se kroz indirektnu komunikaciju putem feromona(Stigimergiju) postigne Swarm intelligence(inteligenciju jata), jer omogućava sistemu da sam pronađe najbolje rešenje i da nastavi čak ako se uslovi promene.

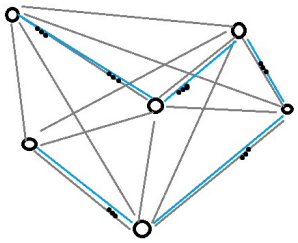
osnovne faze kroz koje prolaze veštacki mravi:

1) Mravi jesu sposobni da sami rade, ali kvalitetne posledice nastaju isključivo zajedničkom iteracijom grupe mrava.
2) Svaki mrav koristi privatne informacije i lokalne ali u okviru čvora u kom se kreće.
3) Mravi komuniciraju isključivo indirektno preko informacija koje čitaju; upisuju promenljive koje čuvaju vrednosti feromona.
4) Mravi se ne menjaju sami za sebe ali menjaju način na koji posmatramo problem
5) Mrav k ima memoriju M^k. On pamti put kojim je išao .
6) Memorija se koristi kako bi se sačuvalo do tada izgradjeno rešenje, kako bi se ono procenilo i kako bi se unazad duž predjenog puta ažurirale vrednosti feromona.
7) Mravu može da se zada početno stanje - čvor od kog polazi i jedno ili više zavsnih stanja il terminalnih uslova
8) Mrav krece od početnog stanja - čvora, kreće se ka dozvoljenim susenim stanjima i korak po korak izgradjuje rešenje. Proces izgradnje se završava kad bar jedan mrav zadovolji bar jedan od terminalnih uslova. Terminalni uslovi su kriterijumi koji odredjuju kada program treba da se završi.
9) Dok se mrav kreće od čvora *i* ka susednom čvoru *j* mrav moze ažurirati trag feromona τ(i, j) na grani (i, j), ovo je onlajn korak-po-korak ažuriranje feromona
10) Kada izgradi rešenje , koristeći memoriju mrav može da se vrati putem kojim je prosao I da ažurira trag feromona na predjenim granama. Ovo je onlajn odloženo ažuriranje feromona.



ACO rešava probleme u računarstvu.

1) **Pronalazi prečice** i brzo otkriva najkraći put od velikog broja opcija.
2) **Sam se popravlja**, jer ako pukne put, mravi odmah nađu drugo rešnje.
3) **Štedi resurse** jer 1000 procesa rade paralelno bez sudaranja tako što koriste inteligenciju umesto teške matematike.
4) **Sprečava greške** : Putem isparavanja feromona program zaboravlja loša ili zastarela rešenja.

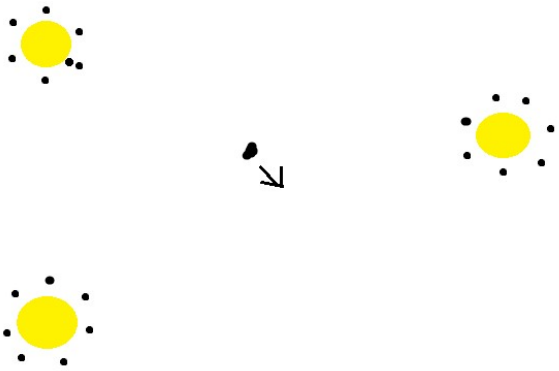


2. Artificial Bee Colony

Model selekcije ABC

Koristi se za pretraživanje ogromnih baza podataka u potrazi za najboljim rezultatom.

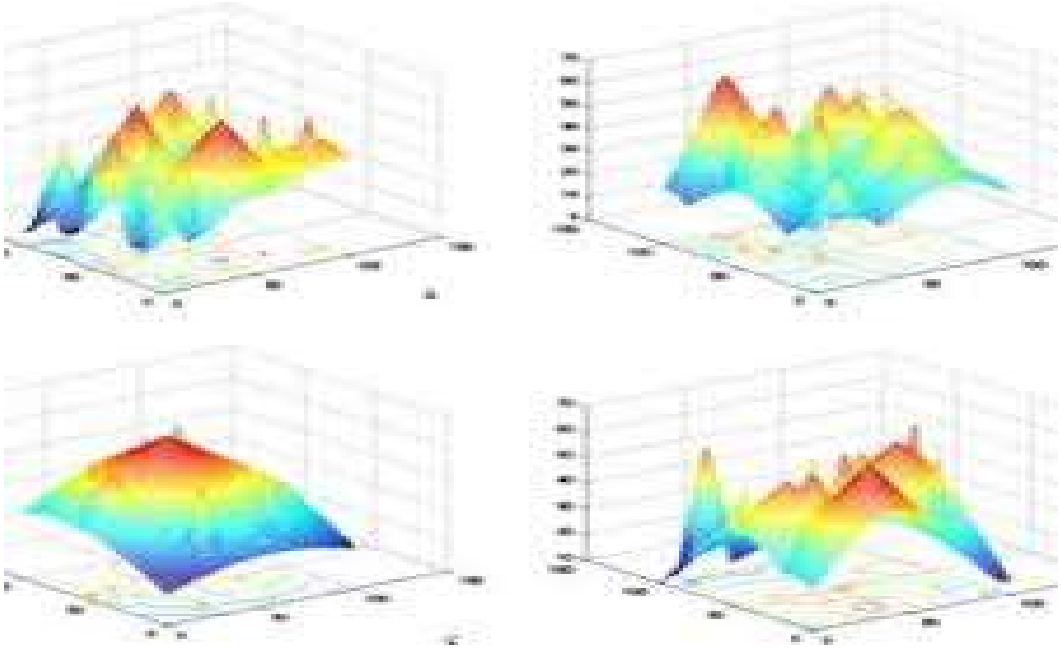
a) Radilice -> sakupljaju izvore hrane i dele ih posmatracima
b) Posmatrači -> prikupljaju izvore hrane koje im prosleđuju radilice preko plesa
c) Izviđači -> razmene i onda traže nove izvore hrane



Cilj: pretraživanje ogromnih baza podataka i optimizacija numeričkih funkcija.

Vrste ABC:

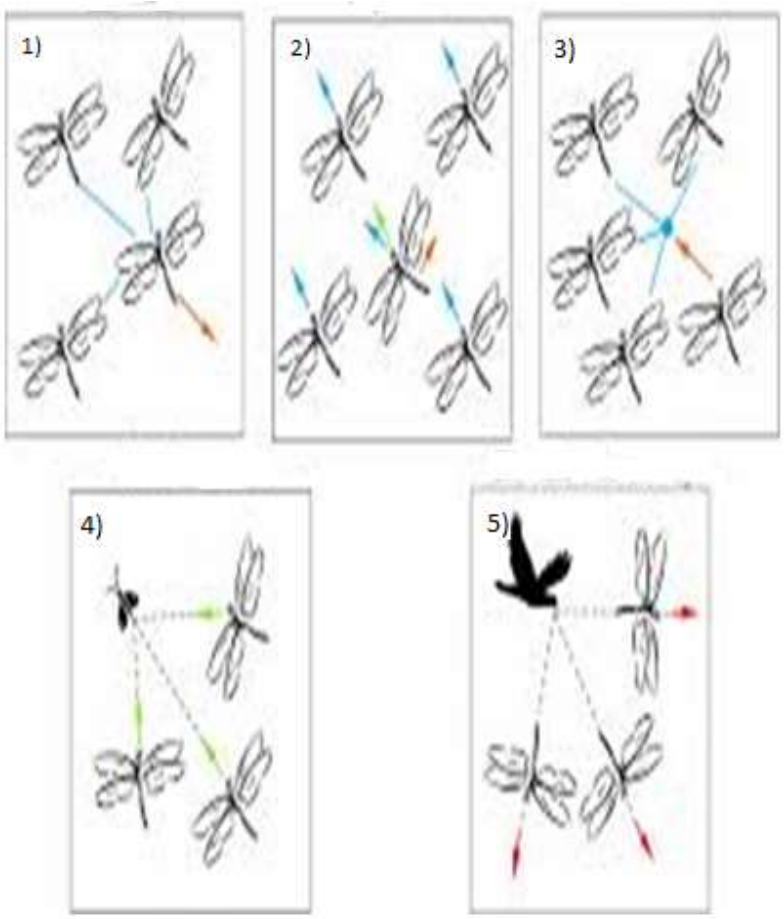
1) Peaks -> navigacija kroz brda i doline, ka najvišem vrhu što je optimalno rešenje problema.
2) Rastrignin->(Izbegavanje zamki)test koji proverava da li algoritam može da izbegne lokalne minimum i pronađe najbolje rešenje.
3) Easom -> traženje igle u plastu sena, povrsina je skoro potpuno ravna osim na 1 strmom mestu.
4) Beale -> test poligon sa ogromnihm ravnim površinama i oštirm ivicama koji proverava da li algoritam može da zadrži brzinu dok traži rešenja.



ABC rešava probleme u računarstvu:

1) Izbegavanjem zamki izviđači sprečavaju računar da se zaglavi u osrednjim situacijama(lokalnom minimumu).
2) Rešavanje teške matematike: Rastrigin I Belae omogućavaju AI da nađu grešku.
3) Pčele štede vreme tako što ignorišu nebitne podatke, i bave se onim koji daju najbolji rezultat.

3. Dragonfly Algorithm



Model kretanja DA

Koristi se za programiranje rojeva malih robota (vilinskih konjica) ili dronova koji NE smeju da se sudare dok obavljaju zadatak.

Dve faze: statički roj (lov/pretraga, male grupe preko uskog prolaza detaljno ispituju teren=eksploatacija) i dinamički roj (migracija, eksploroacija). Izbegavanje sudara i sinhronizaciji brzine.

Cilj : upravljanje rojevima dronova i autonomnim robotima u pokretu.

Vrste DA:

1)Separacija
2)Ponavljane
3)Kohezija
4)Privlačenje ka hrani
5)Odbijanje od neprijatelja

DA rešava probleme u računarstvu:

a)Omogućava dronovima da pretraži teren bez potrebe da čovek upravlja.
b)Koristi se za rešavanje složenih problema gde parametri moraju da se kreću i prilagođuju brzo. Dinamička optimizacija.

4. Firefly Algorithm

Model prati intenzitet svetlosti FA

Privlačenje pomoću svetlosti. Svitac sa slabijim sjajem uvek leti ka onom koji sija jače.

Cilj: grupisanje podataka (clustering) i detekcija objekata na digitalnim slikama.

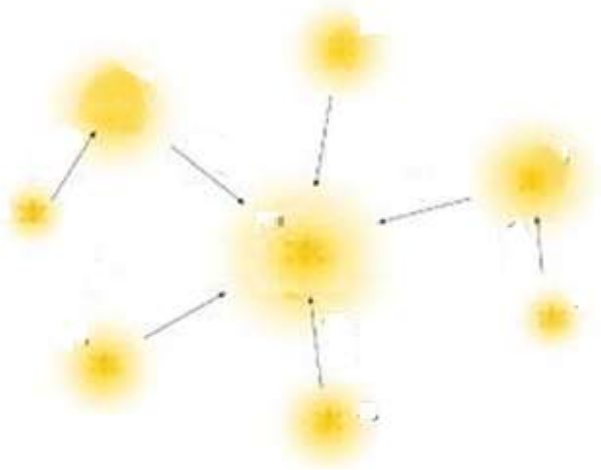
postoje 4 vrste:

1)Rastring -> rešava problem velikog broja lokalnih minimuma.(da Svitac ode na najjače svetlo)

2)Matyas -> testira brzinu konvergencije; preciznost pri cilju.

3)Ackley -> problem strmog centra.(da li ce Svitac detektovati rupu na centru)

4)Peaks -> navigacija kroz brda i doline.



FA rešava probleme u računarstvu :

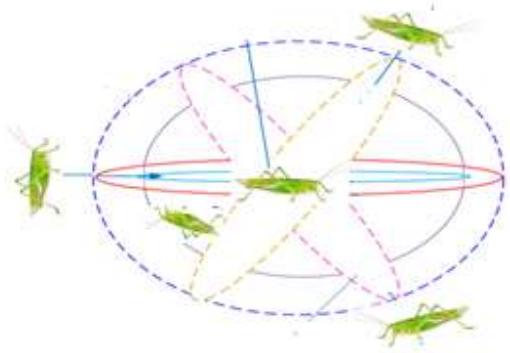
1) Svitac sa slabijim sjajem ide ka jačem, što u računarstvu znači da se slični podaci automatski grupišu I formiraju logičke celine.
2) Koristi se za preciznu detekciju objekata I ivica; jačina svetlosti simulira boje I tako pomaže računaru da vidi gde jedan objekat prestaje, a drugi počinje.
3) 1) i 3) pomažu algoritmu da se ne izgubi na ravnom terenu i da brzo nađe rešenja. Matyas osigurava da ne troši puno vremena.

5. Grasshopper Optimization

Model inteligencije roja

Skakavci, dok su mladi, kreću se malim skokovima (detaljna pretraga), a kao odrasli prave velike skokove (istrazivanje celog prostora).

Cilj: strukturna optimizacija gde je prostor rešenja ogroman.



GOA rešava probleme u računarstvu :

1) Simulira privlačenje I odbijanje među skakavcima, što omogućava računaru da pronađe ravnotežu.
2) Zbog stalne promene načina kretanja, algoritam se retko zaglavi I zato je pouzdan.

6. Bug

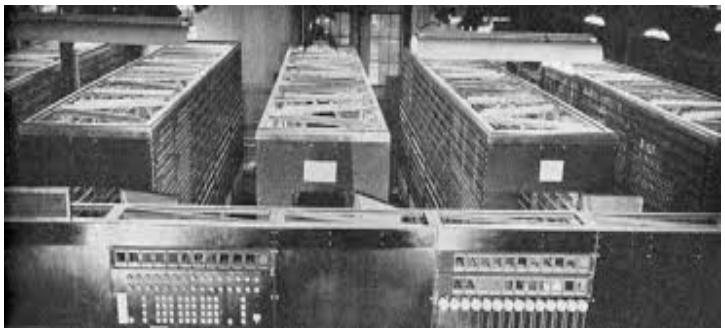
Moljac.

Fizicki ulazak insekta u hardver.

Svaka logička greška u kodu.

Nastala je kad je moljac uleteo veliki računar Harvard

Mark II 1947. izazvao je kratak spoj.



7. Algoritmi

1)ACO:

```
PROCEDURE ACO metaheuristika
    PodesiParametre();
    InicijalizujVrednostTragaFeromona();
    while(nije zadovoljen kriterijum za zaustavljanje) do{
        Rasporedaktivnosti
            KonstrisiMraveIMravljaResenja();
        ->    *PrimeniLokalnuPretragu();
            PodesiVrednostFeromona();
        end RasporedAktivnosti();
    end while
end procedure
```

-> *PrimeniLokalnuPretragu -> verovatno najstarijarija metaheuristika(cilj da nadje ili odabere heuristiku koja je dobro resenje za problem)

```
    PROCEDURE LokalnoPretrazivanje(){
        InicijalizujPocetnoResejne();
        repeat
            for ((svaki korak) do){
            IzaberiOkolinuTrenutnogResenja();
            UNjojNadjiSledeceResenjePremaNekomKriterijumu();
            ZaIzabranoResProveriAkoBoljeOdTrenutnog();
            AkoJesteZameniResNadjenim();
        end for(za svako)
    until(zaustavljanje)
    PrikaziNajboljeNadjenoRes
end PROCEDURE
```

2)ABC

```
        PodesiParametre();
        InicijalizujIzvoreHrane();
        while(nije zadovoljen kriterijum za zaustavljanje) do{
    RadiliceSakupljajuIDele();
        PosmatraciEvoluirajuResenja();
        IzvidjaciTrazeNoveIzvore();
        ZapamtiNajboljeResenje();
```

```
            } end while
        end procedure
```

postoji 4 vrste algoritama:

a) Peaks -> navigacija kroz brda i doline

```
PROCEDURE Peaks
        PodesiParametrePovrsine();
        GenerisiPocetneTacke();
        while(nije pronadjen globalni ekstrem) do{
        IzracunajVrednostFunkcije();
        IdentifikujLokalneVrhoveIDoline();
        AzurirajPozicijuPretrage();
            ProveriKonvergenciju();
        } end while
end procedure
```

b) Rastrignin -> resava problem velikog broja lokalnih minimuma

```
PROCEDURE RastriginOptimizacija
        PodesiParametreMreze();
        InicijalizujNasumicnaResenja();
        while(nije postignuta preciznost resenja) do{
        IzracunajKosinusneOscilacije();
        EvaluacijaFitnesFunkcije();
        IzbegniLokalneMinimume();
        AzurirajNajboljuTacku();
         } end while
end procedure
```

c) Easom -> specijalizovan za pronalazenje veoma uskog globalnog minimuma

```
PROCEDURE Easom
        PodesiParametrePretrage();
        InicijalizujVrednostiXY();
        while(nije pronadjen uski globalni minimum) do{
        IzracunajEksponencijalniPad();
        EvaluacijaKosinusnihProizvoda();
        SužavajZonuPretrage();
        ZapamtiGlobalniMinimum();
```

```
        } end while
end procedure
```

d) Beale -> testira efikasnost optimizacije na ravnim povrsinama sa ostrim ivicama

```
PROCEDURE Beale
        DefinisiGranicePretrage();
        InicijalizujPocetneKoordinate();
        while(nije zadovoljena tolerancija greske) do{
        IzracunajVrednostPolinoma();
        AnalizirajGradijentPovrsine();
        AzurirajPozicijuURavnini();
                MinimizujOdstupanje();
        } end while
end procedure
```

3)DA

```
PROCEDURE DA
InicijalizujPopulacjuRandom(xi, i = 1,2,3, 4....n);
InicijalizujVelicinu(Δxi);
while(TrIter < MaxIter){
        **Computer Fitness vrednosti za svakog vilinskog konjica
        Update (azuriraj trenutne izvore hrane i neprijatelje)
        Update (w, s a, c f, e)
`       07
        Update
        if (vilinK ima bar 1 komsiju vilinK){
                Update velocity & poziciju;
                }
        elif{proveriPoziciju i ispravi trenutnu}
        else{UpdatePoziciju}
end procedure
```

4)FA

```
PROCEDURE FA
InicijalizujPozicijeSvitaca    // u zavisnosti od funkcije svetle
OsvetljenijePrivlaceSlabijeOsvetljene
za svaku iteraciju t1 do T:
```

```
        za svaki par svitaca(i, j):
            if (f(x_j) bolji od f(x_i) pomeri i ka j){
                x_i = x_i + β_o·e^(−γr²)(x_j − x_i) + α·ε
            }
            update bolji
        }end while
    end procedure
```

5)GOA

```
    PROCEDURE GOA
    input
        cMax = maximalnaVrednpst
        cMin = minimalnaVrednost
        l = razmeraPrivlacneDistance
        f = intenzitetPrivlacenja
        n = brojSkakavaca
        MaxIter = maxIteracija
    output
        NajboljePronadjenoResenje

    skakavci -> inicijalizuju n resejna(x1, x2, x3..... xn)
    NadjiNajboljeRes(){
    iter -> 0
    while (iter < MaxIter){
        c = cMax - iter * ((cMax - cMin) / MaxIter);
        for (svaki skakavac i) {
            x novo = c*suma(distanca) + NajboljeRes;
            xi = x novo;
            iter = iter + 1;}
end procedure
```